SwasthyaSetu

End-to-End Development Roadmap — Phase-by-Phase Build Plan

Smart India Hackathon 2026 · Problem Statement 26133 · Government of Maharashtra

Contents

1 Project Snapshot

SwasthyaSetu — *"AI-Assisted Rural Healthcare Coordination & Closed-Loop Referral Platform"* — is a care-coordination platform built to close the biggest gap in rural public healthcare delivery: the referral black hole. Patients in rural Maharashtra move between sub-centres, PHCs, rural hospitals, and district hospitals with no continuity of information and no accountability for whether a referral actually resulted in care.

The platform combines AI-assisted risk triage, facility-centric smart matching, and a closed-loop referral tracking system that follows every patient from initial assessment through to confirmed follow-up completion. It supports two parallel entry routes – Health-Worker-assisted and Patient Self-Service – with an optional Caregiver relationship, and explicitly distinguishes routine from urgent/emergency referral flows so a high-risk case is never left waiting on routine appointment confirmation. AI remains strictly decision support – never diagnosis, prescription, or autonomous clinical decision-making.

Primary Goals

- Ensure every referral is tracked to a definitive outcome (completed, failed, or cancelled) – eliminating the referral black hole.
- Provide AI-assisted risk triage that helps health workers and self-service patients make faster, more consistent decisions without replacing clinical judgment.
- Match patients to the most appropriate facility based on specialty, distance, real-time operational availability, and diagnostic capability – with the facility responsible for internal doctor assignment.
- Ensure a high-risk/emergency case is never trapped waiting on routine appointment confirmation.
- Support a capable patient in navigating the system independently, without becoming an autonomous diagnosis app.

This document sequences the complete build, from Phase 0 (validation) through Phase 24 (future scale), so the team can take the product from a blank repository to a demo-ready, pilot-ready platform.

2 Technology Stack

Layer	MVP Choice	Notes
Mobile	Flutter + Dart	Android-first; one codebase

Layer	MVP Choice	Notes
Admin	Flutter Web or lightweight web client	Same backend APIs
Backend	Java 21 + Spring Boot	Modular monolith
AI	Python + FastAPI	Separate ML service
Database	PostgreSQL, optionally managed through Supabase	Relational source of truth
Local storage	SQLite-based Flutter solution	Protected offline store
Notifications	FCM	Push notifications
Maps	OpenStreetMap + OSRM	With haversine fallback
File storage	S3-compatible storage	MinIO locally or managed storage
Containerization	Docker (optional for local, useful for deployment)	Do not block development
CI/CD	GitHub Actions	Build / test / deploy
Cache	Optional Redis	Add when justified

Supabase does not replace PostgreSQL as the underlying database – a Supabase-based deployment can provide managed PostgreSQL, authentication, storage, and APIs, but the application architecture should still treat PostgreSQL as the relational source of truth.

3 Development Phases

Product: SwasthyaSetu — AI-Assisted Rural Healthcare Coordination & Closed-Loop Referral Platform
Problem Statement ID: 26133
Roadmap Goal: Build a credible MVP first, then harden the system for pilot readiness without over-engineering.

---

4 0. Roadmap Philosophy

SwasthyaSetu should be built in dependency order.

The team should **not** attempt to build every feature simultaneously.

The highest-value sequence is:

```
Foundation
   ↓
Core Patient + Health Worker Workflow
   ↓
AI Safety + Triage
   ↓
Facility Recommendation
   ↓
Closed-Loop Referral
   ↓
Facility Operations + Doctor Assignment
   ↓
Consultation + Follow-up
   ↓
Offline Hardening + Patient Self-Service
   ↓
Admin + Analytics
   ↓
Security + Testing
   ↓
Deployment + Demo
```

The closed-loop referral remains the central differentiator.

---

5 Phase 0 — Product, Workflow & Technical Validation

5.1 Objective

Ensure the team is building the correct system before writing substantial code.

5.2 Tasks

- Freeze the core product vision from `PRD.md` and `MVP.md`.

- Validate the end-to-end referral workflow.
- Define exact responsibilities of:
  - Patient
  - Caregiver relationship
  - Health Worker
  - Facility operations
  - Doctor
  - Admin
- Define routine vs urgent referral behavior.
- Define facility-centric referral semantics.
- Define internal doctor assignment semantics.
- Define offline behavior.
- Define the referral state machine.
- Identify clinical workflows that require domain validation.
- Review existing government systems conceptually.
- Avoid making unsupported claims about ABDM/eSanjeevani.
- Decide final MVP technology choices.

5.3 Technical decisions to freeze

- Flutter
- Spring Boot modular monolith
- FastAPI AI service
- PostgreSQL / Supabase PostgreSQL
- local SQLite-based storage
- FCM
- optional Redis
- optional object storage
- OSM/OSRM with fallback

5.4 Deliverables

- Final workflow diagram
- Role/permission matrix
- Referral state machine
- Initial database ERD
- Architecture decision record
- Final MVP scope

---

6 Phase 1 — Repository & Development Foundation

6.1 Objective

Create a clean, reproducible project foundation.

6.2 Tasks

6.2.1 Repository

Recommended structure:

```
swasthyasetu/
├── apps/
│   ├── mobile/
│   └── admin/
├── services/
│   ├── backend/
│   └── ai/
├── docs/
├── scripts/
└── README.md
```

The exact repository structure may differ, but module boundaries must remain clear.

6.2.2 Flutter

- Initialize Flutter project.
- Material 3.
- Routing.
- Theme.
- Role-aware navigation.
- Environment configuration.
- Basic local persistence abstraction.

6.2.3 Backend

- Initialize Java 21 + Spring Boot.
- Create module packages.
- Configure validation.
- Configure persistence.
- Configure authentication.
- Configure API error format.
- Configure database migrations.

6.2.4 Database

Start with PostgreSQL.

Supabase may be used as the managed PostgreSQL environment, but do not couple the domain layer unnecessarily to Supabase-specific APIs.

6.2.5 AI

- Initialize FastAPI service.
- `/health`
- `/triage` contract.
- Model/rule artifact loading.

6.2.6 Dev environment

- `.env.example`
- local configuration
- basic Docker Compose if useful
- GitHub Actions for test/build

6.3 Deliverable

A developer can clone the repository and run:

```
Flutter
Backend
AI service
Database
```

with documented setup instructions.

---

7 Phase 2 — Authentication, RBAC & Core Data Model

7.1 Objective

Build the security and data foundation before feature logic.

7.2 Tasks

- Authentication.
- JWT/session management.
- User roles.
- Facility scoping.
- Patient identity.
- Patient deduplication.
- Consent capture.
- Caregiver authorization relationship.
- Facility records.
- Specialty records.
- Doctor records.
- Facility capability records.

7.3 Permission tests

Verify that:

- Health Worker cannot access another facility’s patients.
- Doctor cannot access unrelated facility referrals.
- Patient cannot access another patient’s data.
- Caregiver sees only explicitly authorized information.
- Admin sees only permitted jurisdictional information.

7.4 Deliverable

Secure authenticated users can create and retrieve correctly scoped records.

---

8 Phase 3 — Patient + Health Worker Core Workflow

8.1 Objective

Build the most important frontline workflow before adding sophisticated AI.

8.2 Tasks

8.2.1 Health Worker

- Home dashboard.
- Patient registration.
- Duplicate warning.
- Patient search.
- Assessment form.
- Vitals.
- Symptoms.
- Patient history.
- Referral preparation.

8.2.2 Patient

Implement only the patient self-service functionality that is already included in the updated MVP.

Possible initial capabilities:

- basic account/profile,
- own referral/appointment status,
- basic next-action information.

Do not turn this phase into a separate consumer health application.

8.2.3 Caregiver

- Patient-authorized relationship.
- Scoped access.
- Revoke access.

8.3 Deliverable

A Health Worker can:

```
Login
→ Register/find patient
→ Record assessment
→ View patient history
```

---

9 Phase 4 — AI-Assisted Risk Triage

9.1 Objective

Implement safe decision support.

9.2 Tasks

9.2.1 Safety layer first

Implement deterministic rules for:

- invalid vitals,
- defined danger signs,
- incomplete data,
- high-risk escalation conditions.

9.2.2 AI layer

- Feature preparation.
- Baseline model if justified.
- Risk classification.
- Flagged factors.
- Confidence where meaningful.
- Conservative fallback.

9.2.3 API

```
POST /api/triage
```

9.2.4 Human override

- Health Worker/authorized user can override where allowed.
- Mandatory reason.
- Audit event.

9.3 Testing

Test:

- normal input,
- incomplete input,
- dangerous input,
- invalid vitals,
- AI unavailable,
- low-confidence model output.

9.4 Deliverable

Assessment → safe risk stratification works without diagnosis output.

---

10 Phase 5 — Facility Recommendation Engine

10.1 Objective

Build the routing differentiator.

10.2 Tasks

Implement:

- facility capability matching,
- specialty matching,
- diagnostic availability,
- facility operational status,
- distance calculation,
- appointment/availability,
- load,
- freshness of availability data.

10.2.1 Ranking

Implement configuration-driven weights.

10.2.2 Hard filters

Exclude clearly unsuitable facilities before scoring.

10.2.3 Explanation

Every recommendation must provide understandable reasons.

Example:

```
Recommended because:
✓ Required specialty available
✓ Required diagnostics available
✓ Suitable for urgency
✓ 18 km away
```

10.3 External service fallback

If OSRM is unavailable:

```
OSRM → Haversine
```

10.4 Deliverable

Health Worker can see 1–3 appropriate facilities with reasons.

---

11 Phase 6 — Referral State Machine & Closed Loop Foundation

11.1 Objective

Implement the project's single canonical referral state machine as a real, enforced state machine. This is the only referral state-machine definition in the roadmap; no other phase may introduce a competing or overlapping set of referral states.

11.2 Canonical states

TRIAGED is the entry point of the referral lifecycle. There is no separate CREATED referral state — if patient/encounter records are created earlier (Phase 2/3), that is record creation, not a referral status, and it must not be modeled as a referral state.

Implement the full canonical state machine:

- TRIAGED (entry state — assessment complete, AI/rule risk output produced),
- FACILITY_RECOMMENDED (routine path),
- FACILITY_SELECTED,
- FACILITY_CONFIRMATION_PENDING,
- ACCEPTED,
- APPOINTMENT_BOOKED,
- PATIENT_IN_TRANSIT,
- PATIENT_REACHED,
- DOCTOR_ASSIGNED,
- CONSULTATION_COMPLETED,
- DIAGNOSTICS_PENDING → DIAGNOSTICS_COMPLETED (optional branch),
- TREATMENT_COMPLETED,
- FOLLOW_UP_PENDING → FOLLOW_UP_COMPLETED,
- CANCELLED, MISSED_APPOINTMENT, FAILED_REFERRAL (terminal/failure states).

CANCELLED is reachable both before facility selection and after (FACILITY_SELECTED → CANCELLED), e.g. if the patient no longer needs care.

Also implement, as part of this same state machine (not a separate one):

- URGENT_ESCALATION and FACILITY_ALERTED for the emergency branch (TRIAGED → URGENT_ESCALATION → FACILITY_ALERTED → PATIENT_IN_TRANSIT),
- REROUTING_REQUIRED as the single operational-failure state used for facility decline/unavailability/no-eligible-doctor, independent of urgency — both routine and urgent referrals reroute back into FACILITY_RECOMMENDED for re-scoring; if no suitable facility can be found, the referral moves to FAILED_REFERRAL.

Emergency and rerouting are exercised end-to-end in Phase 10; this phase must lay down the states and transition rules so Phase 10 has no gaps to patch by inventing new states.

11.2.1 Critical rule

Do not allow arbitrary status edits.

All transitions must:

- be valid (only the transitions defined above are permitted),
- be authorized,
- create an event,
- record actor/time/reason.

11.2.2 Referral destination

The referral points to a facility (FACILITY_SELECTED / FACILITY_ALERTED).

Do not make an individual doctor the primary destination or a routing key at this stage — doctor assignment happens later, inside Phase 7, strictly after PATIENT_REACHED.

11.3 Deliverable

A referral can move through the full valid lifecycle above (routine and urgent branches) and its complete history can be inspected. No other part of PHASES.md defines a conflicting referral state name or sequence.

---

12 Phase 7 — Facility Operations & Doctor Assignment

12.1 Objective

Make the receiving facility operationally realistic.

12.2 Tasks

12.2.1 Facility

Implement:

- operational status,
- service capability,
- diagnostic availability,
- freshness timestamps,
- basic capacity/load.

12.2.2 Doctor

Implement:

- profile,
- specialty,
- facility association,
- availability,
- leave/unavailable state,
- appointment capacity.

12.2.3 Assignment

Implement:

```
Facility receives referral
→ Required specialty identified
→ Eligible doctors filtered
→ Availability checked
→ Doctor assigned
```

12.2.4 Important behavior

If the originally suitable doctor becomes unavailable:

```
Do not reject the entire referral automatically.
→ Find another eligible doctor.
```

If none exists:

```
Show next available option
OR
Trigger rerouting/escalation according to urgency.
```

12.3 Deliverable

Health Workers think in terms of facility capability; the facility handles doctor assignment internally.

---

13 Phase 8 — Appointment Management

13.1 Objective

Make routine referral scheduling reliable.

13.2 Tasks

- Appointment slots.
- Slot locking/atomic booking.
- Booking confirmation (`ACCEPTED` → `APPOINTMENT_BOOKED`).
- Transition to `PATIENT_IN_TRANSIT` once the patient sets out.
- Rescheduling.
- Cancellation (`CANCELLED`).
- Missed appointment detection (`APPOINTMENT_BOOKED` → `MISSED_APPOINTMENT`) with rebooking back into `APPOINTMENT_BOOKED`.
- Conflict prevention.

13.3 Edge cases

Test:

- slot disappears before confirmation,
- simultaneous booking,
- doctor becomes unavailable,
- facility closes,
- patient cancels,
- patient misses appointment.

13.4 Deliverable

Routine referrals can obtain and maintain a valid appointment without double booking, using the canonical `APPOINTMENT_BOOKED` / `MISSED_APPOINTMENT` states from Phase 6.

---

14 Phase 9 — Doctor Consultation & Care Closure

14.1 Objective

Complete the clinical side of the referral.

14.2 Tasks

14.2.1 Doctor dashboard

- incoming referrals,
- urgency sorting,
- patient summary,
- referral history.

14.2.2 Consultation

- consultation notes,
- outcome,
- transition to `CONSULTATION_COMPLETED`.

14.2.3 Prescription

- medication records,
- prescription generation,
- secure retrieval,
- transition to `TREATMENT_COMPLETED`.

14.2.4 Diagnostics (optional branch)

- order (`CONSULTATION_COMPLETED` → `DIAGNOSTICS_PENDING`),
- result (`DIAGNOSTICS_PENDING` → `DIAGNOSTICS_COMPLETED` → `TREATMENT_COMPLETED`),
- "not required" path (`CONSULTATION_COMPLETED` → `TREATMENT_COMPLETED` directly).

Diagnostics must never be forced on a patient who does not need them; both branches are first-class, not one a fallback of the other.

14.2.5 Follow-up

- schedule date (`TREATMENT_COMPLETED` → `FOLLOW_UP_PENDING`),
- reminder generation,
- follow-up completion (`FOLLOW_UP_PENDING` → `FOLLOW_UP_COMPLETED`),
- expired/unresolved follow-up window → `FAILED_REFERRAL`.

14.3 Deliverable

The complete routine flow works, using the canonical states end to end:

```
TRIAGED
→ FACILITY_RECOMMENDED → FACILITY_SELECTED
→ FACILITY_CONFIRMATION_PENDING → ACCEPTED
→ APPOINTMENT_BOOKED → PATIENT_IN_TRANSIT → PATIENT_REACHED
→ DOCTOR_ASSIGNED
→ CONSULTATION_COMPLETED
→ (DIAGNOSTICS_PENDING → DIAGNOSTICS_COMPLETED, optional)
```

```
→ TREATMENT_COMPLETED
→ FOLLOW_UP_PENDING → FOLLOW_UP_COMPLETED
```

---

15 Phase 10 — Emergency & Rerouting Hardening

15.1 Objective

Ensure the system does not trap urgent cases inside a routine appointment workflow.

15.2 Tasks

Implement, using the states already defined in Phase 6:

- `TRIAGED → URGENT_ESCALATION` for high-risk cases,
- `URGENT_ESCALATION → FACILITY_ALERTED` (nearest capable facility identified; user proceeds without waiting for confirmation),
- `FACILITY_ALERTED → PATIENT_IN_TRANSIT` when the facility can handle the case,
- `FACILITY_ALERTED → REROUTING_REQUIRED` when it cannot,
- Clinical deterioration discovered after `ACCEPTED` is **not** modeled as an `ACCEPTED → URGENT_ESCALATION` transition — it is logged as a `REFERRAL_EVENTS` entry (e.g. `CLINICAL_DETERIORATION_FLAGGED`) that triggers human/facility action,
- `REROUTING_REQUIRED → FACILITY_RECOMMENDED` (applies to both routine and urgent reroutes) and `REROUTING_REQUIRED → FAILED_REFERRAL` (when no suitable facility can be found).

15.2.1 Rerouting is not automatically urgent

`REROUTING_REQUIRED` is an operational-failure state covering: facility unavailable, facility rejects the referral, facility lacks required capability, doctor unavailable, or no suitable doctor/service. It is independent of urgency. A routine referral that fails to find a facility does **not** become `URGENT_ESCALATION` — it re-enters `FACILITY_RECOMMENDED`. Only genuine clinical high-risk triggers `URGENT_ESCALATION`.

15.2.2 Emergency rule

Do not require routine appointment acceptance (`FACILITY_CONFIRMATION_PENDING → ACCEPTED → APPOINTMENT_BOOKED`) before urgent escalation. The urgent branch bypasses routine facility recommendation/selection/confirmation entirely so care is never delayed.

15.2.3 No live availability

If connectivity/live data is unavailable:

- show last known information,
- show freshness,
- warn user,
- allow human decision,
- synchronize later.

15.3 Deliverable

The system has a demonstrable urgent pathway that does not depend on waiting indefinitely for a routine appointment.

---

16 Phase 11 — Offline-First Health Worker Mode

16.1 Objective

Turn offline support into a real product capability rather than a future concept.

16.2 Tasks

16.2.1 Local database

Persist:

- patient data required for current workflow,
- assessments,
- referral drafts,
- referral events,
- essential facility data.

16.2.2 Outbox

Implement:

- local event ID,
- device ID,
- idempotency key,
- retry state.

16.2.3 Sync

Implement:

```
Offline write
→ Queue
→ Connectivity restored
→ Server validation
→ Transaction
→ Server response
→ Local reconciliation
```

16.2.4 Conflict handling

Do not use last-write-wins for:

- referral state,
- appointments,
- doctor assignment,
- emergency escalation.

16.2.5 UI

Display:

- Offline
- Saved locally
- Sync pending
- Synced
- Conflict/action required

16.3 Deliverable

A Health Worker can complete the core capture workflow with no internet and safely synchronize later.

---

17 Phase 12 — Offline Emergency Mode

17.1 Objective

Handle the hardest connectivity scenario.

17.2 Test scenario

```
No internet
→ High-risk patient
→ Assessment saved
→ Temporary referral ID
→ Last-known facility capability
→ Emergency guidance
→ Patient care not blocked by app
→ Internet restored
→ Referral synchronized
→ Facility notified
→ Conflict reconciled
```

17.3 Explicit limitation

The application must never claim live facility acceptance while completely offline.

17.4 Deliverable

A judge can intentionally disable connectivity and observe a safe degradation path.

---

18 Phase 13 — Notifications & Follow-up Reliability

18.1 Objective

Ensure referral continuity does not depend on users constantly checking the application.

18.2 Tasks

Implement:

- in-app notifications,
- FCM push,
- optional SMS adapter,
- referral updates,
- appointment reminders,
- missed appointment alerts,
- follow-up reminders,
- rerouting alerts.

18.3 Failure handling

If push fails:

```
In-app notification remains available.
```

Notifications are not the source of truth.

18.4 Deliverable

Critical events are visible even when an external notification provider fails.

---

19 Phase 14 — Patient Self-Service & Caregiver Experience

19.1 Objective

Strengthen accessibility without replacing the Health Worker model.

19.2 Patient capabilities

Where enabled:

- registration/login,
- basic profile,
- referral status,
- facility information,
- appointment status,
- next action,
- follow-up reminders.

19.3 Caregiver

- request/receive authorization,
- scoped view,
- revoke access.

19.4 Safety

Do not expose:

- raw AI model internals,
- unsupported diagnosis,
- treatment recommendations generated by AI.

19.5 Deliverable

A capable patient can independently understand and follow their care journey, while patients needing assistance can continue using the Health Worker route.

---

20 Phase 15 — Multilingual & Voice Support

20.1 Objective

Reduce language and literacy barriers.

20.2 Tasks

- English.
- Hindi.
- Marathi.
- Language switching.
- Voice input for symptoms.
- Speech-to-text integration.

20.3 Important

Voice is an input method, not a diagnostic engine.

The same AI safety boundaries apply to transcribed text.

20.4 Deliverable

A Health Worker can capture symptoms using supported languages/voice input.

---

21 Phase 16 — Admin Dashboard & Operational Analytics

21.1 Objective

Make the closed-loop impact visible.

21.2 Dashboard metrics

- referral completion rate,
- average referral processing time,
- appointment delay,
- missed appointments,
- failed referrals,
- rerouting frequency,
- follow-up completion,
- facility bottlenecks,
- high-risk follow-up visibility,
- stale facility data,
- AI override rate,
- AI fallback frequency.

21.3 Privacy

Prefer aggregated views.

Patient-identifiable data should be exposed only when necessary.

21.4 Deliverable

Admin can identify where referrals are getting stuck.

---

22 Phase 17 — Security, Privacy & Audit Hardening

22.1 Objective

Move from “works” to “responsible prototype.”

22.2 Tasks

- RBAC penetration checks.
- Facility-scope testing.
- Consent enforcement.
- Caregiver authorization checks.
- Audit log verification.
- Local storage protection.
- Input validation.
- Rate limiting where justified.
- Secure secrets management.
- Token expiry.
- Sensitive logging review.

22.3 Deliverable

Security controls are demonstrably enforced rather than merely documented.

---

23 Phase 18 — Interoperability Readiness

23.1 Objective

Prepare for future integration without pretending it already exists.

23.2 Tasks

Map internal entities conceptually to:

- Patient,
- Observation,
- Encounter,
- ServiceRequest,
- DiagnosticReport,
- MedicationRequest.

Document:

- what a future ABDM integration would require,
- what is not implemented,
- what external authorization would be required.

23.3 Deliverable

A clear interoperability roadmap with no false integration claims.

---

24 Phase 19 — Testing & Reliability

Testing should begin during implementation, but this phase consolidates the full test suite.

24.1 Unit testing

Test:

- referral state machine,
- facility scoring,
- availability filtering,
- doctor assignment,
- triage rules,
- sync logic,
- conflict rules.

24.2 Integration testing

Test:

- Flutter → backend,
- backend → database,
- backend → AI,

- notification adapter,
- sync API.

24.3 API security testing

Test:

- invalid token,
- expired token,
- role bypass,
- facility bypass,
- unauthorized patient access,
- caregiver scope violation.

24.4 End-to-end referral scenarios

Explicitly test the full canonical-state journeys:

- **Routine:** `TRIAGED` → `FACILITY_RECOMMENDED` → `FACILITY_SELECTED` → `ACCEPTED` → `APPOINTMENT_BOOKED` → `PATIENT_IN_TRANSIT` → `PATIENT_REACHED` → `DOCTOR_ASSIGNED` → `CONSULTATION_COMPLETED` → `TREATMENT_COMPLETED`.
- **Emergency:** `TRIAGED` → `URGENT_ESCALATION` → `FACILITY_ALERTED` → `PATIENT_IN_TRANSIT` → `PATIENT_REACHED` → `DOCTOR_ASSIGNED` → `CONSULTATION_COMPLETED`, including emergency failure scenarios (alerted facility cannot handle the case).
- **Rerouting:** facility unavailable/declines/lacks capability → `REROUTING_REQUIRED` → re-enters `FACILITY_RECOMMENDED` for re-scoring (applies to both routine and urgent referrals). Also test the failure path: `REROUTING_REQUIRED` with no suitable facility found → `FAILED_REFERRAL`.
- **Cancellation:** test both `TRIAGED` → `CANCELLED` (withdrawn before facility selection) and `FACILITY_SELECTED` → `CANCELLED` (withdrawn after a facility was already selected, e.g. patient improved).
- **Doctor failure:** `PATIENT_REACHED` with no eligible doctor available → alternative doctor found and `DOCTOR_ASSIGNED`, or (if none exists) `REROUTING_REQUIRED`.
- **Diagnostics required:** `CONSULTATION_COMPLETED` → `DIAGNOSTICS_PENDING` → `DIAGNOSTICS_COMPLETED` → `TREATMENT_COMPLETED`.
- **Diagnostics not required:** `CONSULTATION_COMPLETED` → `TREATMENT_COMPLETED` directly.

24.5 Offline testing

Explicitly test:

1. offline registration,
2. offline assessment,
3. offline referral,
4. reconnect → server confirmation (local save is never treated as server confirmation),
5. duplicate retry,
6. conflicting referral state,
7. stale facility data,
8. emergency offline flow: offline → local emergency record → last-known facility information → stale availability indication → delayed synchronization → no false acceptance confirmation → sync.

24.6 Failure injection

Simulate:

- AI unavailable,
- database temporary failure,
- maps unavailable,
- notification unavailable,
- facility unavailable,
- doctor unavailable.

24.7 AI evaluation

Report actual results.

Never fabricate:

- accuracy,
- sensitivity,
- specificity,
- clinical validation.

24.8 Deliverable

A test report covering functionality, security, offline behavior and AI limitations.

---

25 Phase 20 — Local Demo Stabilization

25.1 Objective

Make the entire MVP run reliably on the team’s MacBook before cloud deployment.

25.2 Tasks

- One-command or clearly documented startup.
- Seed demo facilities.
- Seed demo doctors.
- Seed availability.
- Seed safe demo patient data.
- Verify role accounts.
- Verify complete referral journey.
- Verify offline mode.
- Verify emergency scenario.

25.3 Demo data

Use fictional/synthetic patient information.

Never use real patient information for a hackathon demo unless appropriately authorized and protected.

25.4 Deliverable

A clean local demonstration environment.

---

26 Phase 21 — Deployment & CI/CD

26.1 Objective

Create a reproducible demo/pilot environment.

26.2 Tasks

- Build backend.
- Build AI service.
- Configure database.
- Configure object storage if needed.
- Configure notifications.
- Configure HTTPS.
- Configure secrets.
- Configure backups.
- Configure health checks.
- Configure GitHub Actions.

Docker can be used for deployment reproducibility.

Kubernetes is not required.

26.3 Deliverable

A live deployment with reproducible builds.

---

27 Phase 22 — SIH Demo Preparation

27.1 Objective

Turn the product into a compelling, technically defensible demonstration.

27.2 Primary demo

Use the updated **Meena's Journey**.

Recommended story:

```
Patient arrives
↓
Health Worker records assessment
↓
AI-assisted risk triage
↓
```

```
Facility recommendation
↓
Facility-centric referral
↓
Doctor assignment inside facility
↓
Consultation
↓
Diagnostics / prescription
↓
Follow-up
↓
Referral closed
↓
Admin sees completed journey
```

27.3 Secondary demo

Demonstrate:

```
No internet
↓
Health Worker continues
↓
Emergency/high-risk case
↓
Cached facility information
↓
Urgent escalation
↓
Temporary local referral
↓
Connectivity restored
↓
Safe synchronization
```

27.4 Judge questions to prepare

- Why is this different from eSanjeevani?
- Why facility-centric instead of doctor-centric?
- What happens if a doctor is unavailable?
- What happens if the facility rejects/cannot handle the case?
- How does the system work offline?
- How does it prevent duplicate records?
- What happens when offline data conflicts with server data?
- Why use PostgreSQL/Supabase?
- Why separate the AI service?
- How does AI avoid becoming a diagnosis system?
- What happens if AI fails?
- How would ABDM integration happen?
- What happens if facility availability data is stale?
- What happens in an emergency?

27.5 Deliverable

A rehearsed demo and Q&A script.

---

28 Phase 23 — Pilot Readiness

This is post-MVP/hackathon hardening.

28.1 Tasks

- Real-world workflow validation with authorized stakeholders.
- Usability testing with representative health workers.
- Connectivity testing in realistic environments.
- Clinical/domain review of triage rules.
- Facility data governance.
- Availability update process.
- Privacy/legal review.
- Backup and disaster recovery.
- Monitoring improvements.
- Formal interoperability planning.

28.2 Deliverable

Pilot-readiness report.

---

29 Phase 24 — Future Scale

Explicitly future/post-hackathon.

Potential work:

- district deployment,
- Maharashtra-wide deployment,
- formal ABDM integration,
- eSanjeevani interoperability,
- expanded languages,
- advanced voice interfaces,
- predictive medicine demand,
- aggregated outbreak signals,
- larger analytics platform,
- selective service extraction,
- stronger infrastructure automation.

Nothing in this phase should be presented as already implemented.

---

30 Dependency Map

```
Phase 0  (Validation)
  -> Phase 1  (Foundation)
     -> Phase 2  (Auth + Data)
        -> Phase 3  (Patient + Health Worker)
           -> Phase 4  (AI Triage)
              -> Phase 5  (Facility Recommendation)
                 -> Phase 6  (Referral State Machine)
                    -> Phase 7  (Facility + Doctor Assignment)
                       -> Phase 8  (Appointments)
                          -> Phase 9  (Doctor + Closure)
                             -> Phase 10 (Emergency + Rerouting)
                                -> Phase 11 (Offline)
                                   -> Phase 12 (Offline Emergency)
                                -> Phase 19 (Testing)
                                   -> Phase 20 (Local Demo)
                                      -> Phase 21 (Deployment)
                                         -> Phase 22 (SIH Demo)
                                            -> Phase 23 (Pilot Readiness)
                                               -> Phase 24 (Future Scale)
                             -> Phase 13 (Notifications)
                             -> Phase 16 (Admin)
                                -> Phase 17 (Security)
                                   -> Phase 18 (FHIR Readiness)
           -> Phase 14 (Patient Self-Service + Caregiver)
           -> Phase 15 (Language + Voice)
```

Phase	Depends On	Phase	Depends On
0	–	13	9
1	0	14	3
2	1	15	3
3	2	16	9
4	3	17	16
5	4	18	17
6	5	19	10
7	6	20	19
8	7	21	20
9	8	22	21
10	9	23	22
11	10	24	23
12	11		

31 MVP Cut Line

If development time becomes limited, use this strict priority.

31.1 P0 — Must work

1. Authentication + RBAC
2. Patient registration + deduplication
3. Health Worker assessment
4. AI/rule-based triage
5. Facility recommendation
6. Facility-centric referral
7. Referral state machine
8. Appointment booking
9. Facility/doctor availability basics
10. Doctor assignment
11. Doctor consultation
12. Prescription
13. Diagnostics
14. Follow-up
15. Referral closure
16. Basic admin dashboard

31.2 P1 — Strongly preferred / pilot-readiness

1. Offline-first Health Worker mode
2. Offline emergency mode
3. Rerouting
4. Facility freshness/availability
5. Notifications
6. Hindi/Marathi
7. Voice input
8. Patient self-service
9. Caregiver relationship
10. Enhanced facility operational controls

31.3 P2 — Future / stretch

1. Live ABDM integration
2. eSanjeevani interoperability
3. Advanced predictive analytics
4. Expanded multilingual voice
5. Large-scale infrastructure
6. Advanced operational optimization

---

32 Parallel Development Strategy

The phases are ordered by dependency, but some work can happen in parallel after the relevant contracts are frozen.

```
Backend team:
Auth → Data → Referral → Facility → Doctor → Admin

AI team:
Rule safety → Triage API → Evaluation
```

```
Flutter team:
Role shell → HW workflow → Doctor workflow → Patient workflow → Offline

Infrastructure:
DB → CI → Environments → Deployment

Testing:
Begin with every completed module
```

Do not wait until the end to test.

---

33 Definition of Done

A phase is not complete merely because the code compiles.

A phase is complete when:

- functionality works,
- API authorization is tested,
- failure behavior is defined,
- relevant audit events exist,
- UI handles errors,
- data is persisted correctly,
- documentation matches implementation,
- tests exist for critical logic.

---

34 Final Build Order

For actual implementation, the team should follow:

```
1. Repository + environment
2. Database + migrations
3. Auth + RBAC
4. Patient
5. Health Worker
6. Assessment
7. Rule-based triage
8. AI service
9. Facility data
10. Facility recommendation
11. Referral state machine
12. Appointment
13. Facility availability
14. Doctor availability
15. Doctor assignment
16. Doctor dashboard
17. Consultation
18. Diagnostics
19. Prescription
```

```
20. Follow-up
21. Emergency/rerouting
22. Offline sync
23. Offline emergency
24. Notifications
25. Patient self-service
26. Caregiver access
27. Multilingual/voice
28. Admin dashboard
29. Security hardening
30. Testing
31. Local demo
32. Deployment
33. SIH demo
```

---

35 Final Roadmap Principle

The project should never become “feature-complete but unreliable.”

The priority is:

> **A smaller system that reliably closes a referral loop is more valuable than a huge system that only creates referrals.**

The core proof remains:

```
Need identified
→ Risk assessed safely
→ Right facility selected
→ Facility can actually handle the case
→ Doctor/service assigned
→ Patient reaches care
→ Consultation/treatment recorded
→ Follow-up tracked
→ Outcome visible
```

That is the system the architecture and development roadmap are designed to build.